| CL1002<br>***Programming Fundamentals Lab*** | Lab 12<br>Introduction to Pointers and Dynamic Memory Allocation |
|---|---|

**NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES**

Fall 2024

## AIMS AND OBJECTIVES

To understand the concepts of pointers and dynamic memory allocation in C/C++ and apply them to create, manipulate, and free memory dynamically during runtime.

## INTRODUCTION

Pointers and dynamic memory allocation are fundamental concepts in C and C++ programming that provide programmers with powerful tools for efficient memory management and flexibility. Unlike static memory allocation, where the size of memory is determined at compile-time, dynamic memory allocation allows the program to request and manage memory at runtime based on actual requirements.

Pointers, which store the memory address of another variable, are at the core of dynamic memory allocation. By leveraging pointers, developers can create data structures of varying sizes, manage large datasets, and optimize the usage of system memory. However, with great power comes responsibility: improper use of pointers and dynamic memory can lead to errors such as memory leaks, segmentation faults, and undefined behavior.

## POINTER DECLARATION

With the theory in place lets look at some examples.

Pointer Declaration:

```
1   #include <stdio.h>
2
3   int main() {
4       int a = 10;
5       int *p;
6       p = &a;
7       printf("Value of a: %d\n", a);
8       printf("Address of a: %d\n", &a);
9       printf("Value of p (Address stored in pointer):
          %d\n", p);
10      printf("Value pointed to by p: %d\n", *p);
11
12      return 0;
13  }
```

**Note:**

**&** (Address-of operator): Gets the memory address of a variable.
Example: p = **&**a; assigns the address of a to p.
***** (Dereference operator): Accesses the value at the memory address stored in the pointer.
Example: *p gives the value of a in this case.

Pointers and Arrays:
Pointers and arrays are closely related. Here's an example to illustrate:

```
1   #include <stdio.h>
14
15      int main() {
16          int arr[] = {10, 20, 30};
17          int *p = arr;
18          printf("Address of arr[0]: %d, Value:%d\n",p, *p);
19          printf("Address of arr[1]: %d, Value:%d\n",
            (p + 1), *(p + 1));
20          printf("Address of arr[2]: %d, Value: %d\n",
            (p + 2), *(p + 2));
21
22          return 0;
23      }

24
```

Note:
When declaring an array to a variable it's a reference to memory which is the first block of the contagious memory. Adding/Subtracting into this reference changes the starting point of the array.

Null Pointers:
Pointers which point to nothing. They exist as placeholders for later use.

```
1   #include <stdio.h>
2
3   int main() {
4       int *p = NULL;    // Null pointer, points to nothing
5
6       if (p == NULL) {
7           printf("Pointer is NULL, it does not point to any
                valid memory location.\n");
8       }
9
10      return 0;
11  }
```

## 2D POINTERS

Previously, we discussed how arrays are essentially pointers to memory blocks, but what about 2-dimensional pointers or an array of pointers.

**Concept 1: Pointer to a 2D Array**

A 2D array is essentially an array of arrays stored in contiguous memory. A pointer can be used to traverse and manipulate this memory.

```
1   #include <stdio.h>
25
26      int main() {
27          int arr[2][3] = { {1, 2, 3}, {4, 5, 6} };
28          int (*p)[3];
29
30          p = arr;
31
32          printf("Accessing elements using the pointer:\n");
33          printf("arr[0][0]: %d, arr[0][1]: %d, arr[0][2]:
                %d\n", (*p)[0], (*p)[1], (*p)[2]);
34          printf("arr[1][0]: %d, arr[1][1]: %d, arr[1][2]:
        %d\n", (*(p + 1))[0],(*(p + 1))[1],(*(p + 1))[2]);
35
36          return 0;
37      }
```

**Explanation**

1. **int (*p)[3];**: Declares a pointer to an array of 3 integers.
2. **p = arr;**: Assigns the base address of the first row of the 2D array to p.
3. Use *p to access rows and (*p)[col] to access individual elements.

**Concept 2: Array of Pointers**

In some cases, you may want to store a list of arrays, each of varying lengths. This is achieved using an array of pointers.

```
1  #include <stdio.h>
2
3  int main() {
4      int row1[] = {1, 2, 3};
5      int row2[] = {4, 5};
6      int row3[] = {6, 7, 8, 9};
7
8      int *arr[] = {row1, row2, row3}; // Array of pointers
9
10     printf("Accessing elements from the array of
               pointers:\n");
11     printf("arr[0][2]: %d\n", arr[0][2]);
12     printf("arr[1][1]: %d\n", arr[1][1]);
13     printf("arr[2][3]: %d\n", arr[2][3]);
14
15     return 0;
16 }
```

**Explanation**

1. **int *arr[] = {row1, row2, row3};**: Declares an array of pointers where each pointer points to a row.
2. Use arr[row][col] syntax for direct access to elements.

## VOID POINTERS

A **void pointer** is a special type of pointer that can point to any data type. This is in contrast to normal data type pointers, which are bound to point to specific data types like int, float, or char.

Declaration of Void Pointers:

```
1  void *ptr;
```

Note:

**Generic Pointers**: These pointers are called Generic Points and can hold the address of any data type.

**Typecasting Required**: Since the type is not known, void pointers cannot be dereferenced directly; you must typecast them to the appropriate type first.

**Memory Management**: Often used in dynamic memory allocation functions like `malloc()` and `free()`

## EXAMPLE

```
1  #include <stdio.h>
38
39      int main() {
40          int a = 10;
41          float b = 5.5;
42          char c = 'X';
43
44          void *ptr; // Declare a void pointer
45
46          // Point to different data types
47          ptr = &a;
48          printf("Value of a using void pointer: %d\n",
              *(int *)ptr);
49
50          ptr = &b;
51          printf("Value of b using void pointer: %.2f\n",
             *(float *)ptr);
52
53          ptr = &c;
54          printf("Value of c using void pointer: %c\n",
            *(char *)ptr);
55
56          return 0;
57      }
```

**When to Use Void Pointers?**

**Generic Functions**: Functions like malloc() and free() return void * because they don't depend on the type of data being allocated or freed.

```
1   int *p = (int *)malloc(5 * sizeof(int));
```

**Generic Data Structures**: For example, when implementing linked lists, stacks, or queues that can store any type of data.

## EXAMPLE

```
1   #include <stdio.h>
58
59      void printValue(void *ptr, char type) {
60          switch (type) {
61              case 'i': printf("Integer: %d\n", *(int
                    *)ptr); break;
62              case 'f': printf("Float: %.2f\n", *(float
                        *)ptr); break;
63              case 'c': printf("Character: %c\n", *(char
                        *)ptr); break;
64              default: printf("Unknown type\n");
65      }  }
67
68      int main() {
69          int a = 42;
70          float b = 3.14;
71          char c = 'A';
72
73          printValue(&a, 'i');
74          printValue(&b, 'f');
75          printValue(&c, 'c');
76
77          return 0;
78      }
```

## DYNAMIC MEMORY ALLOCATION (DMA)

Dynamic Memory Allocation (DMA) allows you to allocate memory during runtime, providing flexibility to create data structures like arrays or linked lists when the size is not known at compile time. This is particularly useful when dealing with varying data sizes or optimizing memory usage.

There are two types of memory allocation in C.

**Compile-Time Memory Allocation (Static Allocation):**

- Memory is allocated for variables and data structures at compile time.
- The size is fixed and cannot change during execution.
- Done in:
  - **Global Memory**: For global and static variables.
  - **Stack Memory**: For local variables and function call-related data.

**Example**:

```
1 int a = 10;  // Compile-time allocation
2 int arr[5];  // Array of fixed size
```

**Run-Time Memory Allocation (Dynamic Allocation):**

- Memory is allocated during execution using functions like malloc, calloc, and realloc.
- The size and lifetime of the memory are determined at runtime, offering flexibility.
- Managed in the **Heap Memory**.

**Example:**

```
1 int *ptr = (int *)malloc(sizeof(int) * 5);  // Runtime allocation
```

Side-by-Side comparison

| Aspect | Compile-Time Allocation | Dynamic Memory Allocation |
|---|---|---|
| When allocated | At compile time. | At runtime. |
| Memory Source | Stack or global memory. | Heap memory. |
| Size Flexibility | Fixed size (must be known at compile time). | Can change during execution using realloc. |
| Management | Automatically managed by the compiler. | Must be explicitly managed by the programmer. |
| Lifetime | Limited to scope for local variables or static for globals. | Persists until explicitly freed using free. |
| Examples | int a[10];, int x = 20; | int *p = (int *)malloc(10 * |

| | | |
|---|---|---|
| | | sizeof(int)); |

**Static vs Dynamic Allocation in a Program**

Compile-Time (Static Allocation) Example:

```
#include <stdio.h>

    int main() {
        int arr[5];  // Fixed array, size cannot change

        for (int i = 0; i < 5; i++) {
            arr[i] = i + 1;
            printf("%d ", arr[i]);
        }

        // Cannot add more elements dynamically
        return 0;
    }
```

Dynamic Memory Allocation Example:

```
#include <stdio.h>
    #include <stdlib.h>

    int main() {
        int n;
        printf("Enter the number of elements: ");
        scanf("%d", &n);

        int *arr = (int *)malloc(n * sizeof(int));
        if (arr == NULL) {
            printf("Memory allocation failed!\n");
            return 1;
        }

        for (int i = 0; i < n; i++) {
            arr[i] = i + 1;  // Initialize elements
            printf("%d ", arr[i]);
        }

        free(arr);  // Free the allocated memory
        return 0;
    }
```

Note: In the static example the size of the array is fixed, while in the dynamic example the size of the array is determined at runtime.

**Question to you :**

What if I try to allocate the memory like this

```
1 int n;
2 scanf(“%d”,&n);
3 int arr[n]
```

Is this a compile time memory allocation or runtime memory allocation?

Methods to allocate memory in C:

There exists three primary ways to allocate memory in runtime in C.

**malloc (Memory Allocation)**

- Allocates a **single block of memory** of the specified size (in bytes).
- Memory is **not initialized** (contains garbage values).
- Returns a pointer to the allocated memory or NULL if allocation fails.

```
1 int *ptr = (int *)malloc(5 * sizeof(int)); // Allocates memory for 5 integers
2 if (ptr == NULL) {
3     printf("Memory allocation failed!\n");
4 }
```

**calloc (Contiguous Allocation)**

- Allocates memory for an **array of elements**, each of a specified size.
- Memory is **initialized to zero**.
- Useful for initializing arrays or buffers.

```
1 int *ptr = (int *)calloc(5, sizeof(int)); // Allocates and initializes memory for 5
  integers
2 if (ptr == NULL) {
3     printf("Memory allocation failed!\n");
4 }
```

**realloc (Reallocation)**

- Resizes a previously allocated memory block (using malloc or calloc).
- Can increase or decrease the size of the memory block.
- If the new size is larger:
    - Data in the original block is preserved.
    - The new portion remains uninitialized.
- If reallocation fails, it returns NULL without affecting the original block.

```
1 int *ptr = (int *)malloc(5 * sizeof(int));
```

```
    ptr=(int*)realloc(ptr,10*sizeof(int)); //Resize to 10
    if (ptr == NULL) {
        printf("Reallocation failed!\n");
    }
```

**MALLOC EXAMPLE**

```
#include <stdio.h>
    #include <stdlib.h>

    int main() {
        int *ptr;
        int n, i;

        printf("Enter the number of elements: ");
        scanf("%d", &n);

        // Dynamically allocate memory using malloc
        ptr = (int *)malloc(n * sizeof(int));

        if (ptr == NULL) {
            printf("Memory allocation failed!\n");
            return 1;
        }

        // Input elements
        printf("Enter %d integers:\n", n);
        for (i = 0; i < n; i++) {
            scanf("%d", &ptr[i]);
        }

        // Print elements
        printf("You entered:\n");
        for (i = 0; i < n; i++) {
            printf("%d ", ptr[i]);
        }

        // Free the allocated memory
        free(ptr);

        return 0;
    }
```

## CALLOC EXAMPLE

```
1   #include <stdio.h>
150     #include <stdlib.h>
151
152     int main() {
153         int *ptr;
154         int n, i;
155
156         printf("Enter the number of elements: ");
157         scanf("%d", &n);
158
159         // Dynamically allocate memory using calloc
160         ptr = (int *)calloc(n, sizeof(int));
161
162         if (ptr == NULL) {
163             printf("Memory allocation failed!\n");
164             return 1;
165         }
166
167         printf("Memory initialized to zero:\n");
168         for (i = 0; i < n; i++) {
169             printf("%d ", ptr[i]);
170         }
171
172         free(ptr);
173
174         return 0;
175     }
```

## REALLOC EXAMPLE

```
1   #include <stdio.h>
176     #include <stdlib.h>
177
178     int main() {
179         int *ptr;
180         int n1, n2, i;
181
182         printf("Enter the initial number of elements: ");
183         scanf("%d", &n1);
184
185         ptr = (int *)malloc(n1 * sizeof(int));
186
187         if (ptr == NULL) {
188             printf("Memory allocation failed!\n");
189             return 1;
190         }
191
192         printf("Enter %d integers:\n", n1);
193         for (i = 0; i < n1; i++) {
194             scanf("%d", &ptr[i]);
195         }
196
197         printf("Enter the new size of the array: ");
198         scanf("%d", &n2);
199
200         // Resize the memory block using realloc
201         ptr = (int *)realloc(ptr, n2 * sizeof(int));
202
203         if (ptr == NULL) {
204             printf("Memory reallocation failed!\n");
205             return 1;
206         }
207
208         if (n2 > n1) {
209             printf("Enter %d more integers:\n", n2 - n1);
210             for (i = n1; i < n2; i++) {
211                 scanf("%d", &ptr[i]);
212             }
213         }
214
215         printf("Updated array:\n");
216         for (i = 0; i < n2; i++) {
217             printf("%d ", ptr[i]);
218         }
219
220         free(ptr);
221
222         return 0;
223     }
```

Note: The free() function is to deallocate memory when its functionality is no longer required to avoid memory leaks. Notice in the above-mentioned example There exists dangling pointers and this should be set to NULL otherwise undefined behavior can occur.

Example:
Consider this code

```
1   int *ptr = (int *)malloc(sizeof(int));
224     *ptr = 10;
225     free(ptr);  // Memory is freed, ptr is now dangling
226     *ptr=20; // Undefined behavior: accessing freed memory
```

Because the pointer being free of the allocated memory cannot act as a traditional reference to a variable we would need to set the pointer to NULL so we can use it as intended.

**Double Pointers:-**

A double pointer (denoted as **) is a pointer to another pointer. It stores the address of a pointer variable. This allows you to create multiple levels of indirection, which is useful in scenarios like dynamic memory allocation, multi-dimensional arrays, or modifying pointers within functions.

Syntax:

int **ptr; // ptr is a pointer to a pointer to an integer.

**Example:-**

```
#include <stdio.h>

void addOne(int **ptr) {

// Increment the value of the variable indirectly through double pointer

**ptr += 1;

}

int main() {

int num = 5; // Initial variable

int *p = &num; // Pointer to num

int **pp = &p; // Double pointer pointing to p

printf("Before: num = %d\n", num);
```

```
addOne(pp); // Pass double pointer to the function

printf("After: num = %d\n", num);

return 0;

}
```

# Problems

Q1: You are tasked with creating a C program to manage student grades dynamically. The program should allow users to input the number of students and, for each student, specify the number of grades and enter those grades. Once the data is entered, the program should display the grades for each student.

Implement dynamic memory allocation to handle a flexible number of students and varying numbers of grades per student.
Prompt users to input the total number of students.
For each student, prompt users to input the number of grades and the actual grades.
Display the entered grades for each student.
Ensure proper memory management by freeing allocated memory after use.

**Output**

```
Enter the number of students: 3
Enter the number of grades for student 1: 3
Enter grades for student 1:
  Grade 1: 12
  Grade 2: 34
  Grade 3: 78
  Enter the number of grades for student 2: 3
  Enter grades for student 2:
  Grade 1: 90
  Grade 2: 45
  Grade 3: 50
  Enter the number of grades for student 3: 3
  Enter grades for student 3:
  Grade 1: 12
  Grade 2: 90
  Grade 3: 1
  Entered Grades:
Student 1 Grades: 12 34 78
Student 2 Grades: 90 45 50
Student 3 Grades: 12 90 1
```

Q2: Your function is given an array containing some numbers. Decrease the value of all odd numbers in the array by 1, so that the entire array contains only even numbers. Input array using malloc.

Q3: Write a C program to create and manage a dynamic array of floating-point numbers using pointers and dynamic memory allocation. The program should perform the following operations:

1. Dynamically allocate memory for an array of floats with an initial capacity of 4 elements.
2. Allow the user to enter floating-point numbers into the array. If the array becomes full, *double its capacity* using `realloc` to accommodate more elements.
3. Provide the following functionality:
    - **a.** Add a new number to the array.
    - **b.** Display all numbers in the array.
    - **c.** Remove the last number from the array (if not empty).
    - **d.** Reduce the memory size to fit the current number of elements if the array's usage drops significantly.
    - **e.** Free the allocated memory before the program terminates.

Q4: Given a list of integers, find out the number that has the highest frequency. If there are one or more such numbers, output the smaller one.

***Input***

First line of input will contain an integer T = number of test cases. Each test case will contain two lines. First line will contain an integer N = number of elements in the sequence and 1 <= N <= 1000. Next line will contain N space separated integers of sequence A. For each Ai in sequence A, 1<= Ai <= 10001.

***Output***

```
3
7
2 4 5 2 3 2 4
6
1 2 1 1 2 1
10
1 1 1 1 1 1 1 1 1 1
```

***Sample Output***

```
2
1
1
```

Q5:Given a list of integers allocated dynamically ,perform insertion & deletion on an array on the given position. Reallocate array with size greater than the initialized if size is full.

***Input***

First line of input will contain an integer T = number of test cases. Each test case will contain two options(insertion & deletion). First line will contain an integer N = number of elements in the sequence and 1 <= N <= 1000. Next line will contain N space separated integers of sequence A. For each Ai in sequence A, 1<= Ai <= 10001.

```
7

3

I 8 23

D 5
```

***Output***

```
15 7 3 6 2 44 2

15 7 3 6 2 44 2 23

15 7 3 6 44 2 23
```

Q6: Design a C program using dynamic memory allocation to find the largest element in a collection of user-inputted values. Begin by prompting the user to input the total number of elements (ranging from 1 to 100). Subsequently, request input for each element individually, and store them dynamically. Implement the necessary logic to identify the largest element among the entered values. ensure proper memory management by deallocating memory after use.

Q7:- Write a program that uses a double pointer to dynamically allocate memory for a 2D array, where the number of rows and columns is determined at runtime. The program should initialize the array with values, modify one of the values through the double pointer, and then print the entire array. Finally, ensure all allocated memory is properly freed using the double pointer to avoid memory leaks.

Q8: Design a C program that manages point fee details for a sports facility. Users input the total number of points, and for each point, the associated fee is collected. The program calculates the total profit generated by summing up all fees. Error handling ensures negative fees are addressed appropriately. Provide a code highlighting the steps, including dynamic memory allocation, user input, profit calculation, and proper memory management.

```
Enter the total number of points in the facility: 5
Enter fee for point 1: $4500
Enter fee for point 2: $6565
Enter fee for point 3: $9000
Enter fee for point 4: $1200
Enter fee for point 5: $890
Total Profit for the facility: $22155.00
```